

Engenharia de Requisitos e Processos de *Software*



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Técnicas e Ferramentas da Engenharia de Requisitos

Responsável pelo Conteúdo:

Prof. Me. Artur Marques

Revisão Textual:

Prof. Me. Claudio Brites

UNIDADE

Técnicas e Ferramentas da Engenharia de Requisitos



- Introdução;
- Atividades e Técnicas da Engenharia de Requisitos;
- Requisitos como Histórias do Usuário;
- Gerenciamento;
- Documentação;
- Rastreabilidade de Requisitos;
- Matriz de Rastreabilidade de Requisitos;
- Diagrama de Caso de Uso.



OBJETIVO DE APRENDIZADO

- Utilizar as técnicas de levantamento de requisitos.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Faremos uma recordação breve sobre requisito funcional, porque ele acaba sendo preponderante nas eliciações de requisitos em geral e quando utilizamos ferramentas para rastreamento e tabelas de referências de requisitos, sendo sempre a estrela principal.

Conforme escritos de Ventura (2016, p. 1):

É comum os profissionais de engenharia de *software* associarem a ideia de um requisito funcional a uma tela, uma rotina, que no fim serão as funcionalidades de fato de um sistema. Mas é necessário entender que uma funcionalidade não necessariamente realizará apenas um Requisito Funcional, mas talvez vários.

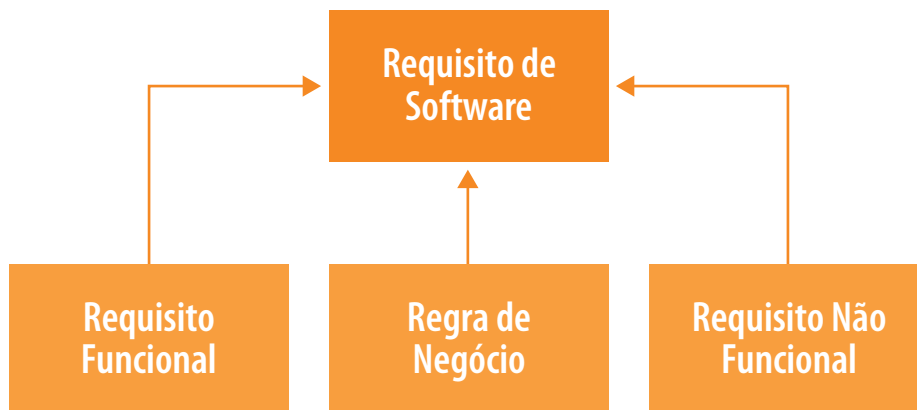


Figura 1 – Um Requisito Funcional compõe um Requisito de *Software*, junto com as regras de negócio e os requisitos não funcionais

Conforme os escritos canônicos de Sommerville (2011), as fontes de requisitos podem ser de três tipos:

- **Stakeholders:** são a maior fonte de requisitos. Pessoas ou organizações que têm influência (direta ou indireta) nos requisitos, como usuários, clientes, desenvolvedores;
- **Documentos:** norma e legislações que podem delimitar o campo de ação do projeto ou documentos específicos da organização (por exemplo, relatórios de falhas de sistemas anteriores);
- **Sistemas em operação:** sistemas anteriores ou atualmente em uso podem ensinar muito sobre o processo, assim como sistemas similares de concorrentes, por exemplo. A partir da sua análise de uso, podemos pensar em extensões ou alterações para otimizá-lo. (SOMMERVILLE, 2011, p. 65-69)

Atividades e Técnicas da Engenharia de Requisitos

Elicitação

Essa atividade também é conhecida como captura ou levantamento de requisitos. É o processo de gerar uma lista de requisitos funcionais, de sistema, técnicos etc. a partir da necessidade das várias partes interessadas – por exemplo: clientes, usuários, fornecedores, equipe de TIC, financiadores, entre outros –, itens listados que serão usados como base para o documento formal de requisitos.

Elicitar é um processo sofisticado e que, ao contrário do que muitos pensam, inclusive da própria área de TIC, não é tão simples como “sair pegando usuários e fazer perguntar às partes interessadas, ou o que elas querem que o sistema faça”, pois, em muitos casos, elas não estão cientes de todas as possibilidades que existem e podem ser limitadas por sua imersão no estado atual de suas rotinas.

Portanto, há uma série de técnicas úteis que você deve dominar e saber empregar conforme a demanda apresentada:

- **Entrevistas:** É uma das ferramentas mais utilizada no início do processo para obter informações básicas sobre os problemas do negócio e compreender, em uma perspectiva do mundo atual, o que o sistema que está sendo proposto precisa fazer;
- **Questionários:** Também muito utilizados, porém, um dos desafios aqui é que você só receberá as informações que a pessoa está manipulando no dia a dia e, normalmente, aquilo que é mais urgente de se resolver na vida de trabalho dela, portanto, problemas; o restante fica em segundo plano ou até mesmo acaba sendo esquecido. Um dos resultados nos questionários é que eles não são muito bons para aqueles requisitos latentes. Numa entrevista baseada em um questionário inicial, podemos sondar com base nas informações capturadas detalhes de áreas específicas que as partes interessadas não sabem que são importantes, mas que podem se mostrar absolutamente críticas para o design final da solução;
- **Observação do usuário ou etnografia:** Ferramenta de melhoria e serve para observar os usuários executando suas tarefas diárias e, idealmente, registrando as ações e atividades que ocorrem. Compreendendo o contexto de como eles executam as tarefas, você pode escrever requisitos que reinventarão os processos, em vez de apenas automatizá-los;
- **Workshops:** Após definir o conjunto de possíveis requisitos, você precisará consultar e harmonizar as opiniões e visões divergentes para garantir que o sistema atenda às necessidades de todos os usuários, e não apenas do grupo

mais influente. São muito úteis para obter consenso e capturar as premissas e restrições do processo em análise para o futuro sistema;

- **Brainstorming:** É uma ferramenta bastante poderosa quando bem-feita e com um líder que controle a atividade. Isso porque, ao considerar diferentes partes do sistema e cenários hipotéticos, ou ideias originadas do livre pensar das pessoas que participam do *brainstorming*, você pode sair do contexto do estado atual e considerar ideias visionárias para o futuro;
- **Role-playing games (RPG) (jogo de papéis e regras):** Em situações em que os requisitos dependem fortemente de diferentes tipos de usuários, o RPG, em que pessoas diferentes assumem os papéis de diferentes usuários no processo, pode ser uma boa maneira de entender como as diferentes partes do futuro sistema precisam trabalhar para apoiar a integração e a sinergia nos processos. Isso é bom porque obriga determinados usuários a “vestirem a pele de outros”;
- **Casos de uso e cenários:** Trata-se de uma ferramenta extremamente influente nos dias atuais, junto com a entrevista e prototipagem, porque, depois de definir os requisitos funcionais de alto nível, é fundamental começar a desenvolver diferentes diagramas de casos de uso e cenários que possam ser usados para validar a funcionalidade em diferentes situações;
- **Prototipagem:** Durante os exercícios de sua profissão como analista, por diversas vezes, as partes interessadas não têm uma ideia clara sobre quais são os requisitos. A solução elegante para isso é reunir vários protótipos diferentes mostrando como serão ou, ao menos, poderiam ser as telas, as entradas de dados, a dinâmica funcional ou a usabilidade do futuro sistema, para ver se os usuários gostam de tudo, ou de que partes eles gostam. Assim, você pode sintetizar as diferentes partes que eles gostaram dos protótipos e puxar os requisitos a partir deles. Claro, você também aproveitará para utilizar esse resultado no desenvolvimento do sistema, principalmente no que tange à camada de apresentação, onde há o desenvolvimento da interface gráfica do usuário.

Requisitos como Histórias do Usuário

Uma história de usuário é uma forma de elicitar requisitos de sistema que se tornou bastante popular em metodologias ágeis. A ênfase aqui é a simplicidade e a permutabilidade, portanto, as histórias são projetadas para serem facilmente descritas e compreendidas e, mais importante, facilmente alteradas pelo cliente final durante o projeto. Nesse caso, e por se tratar de metodologia ágil, o cliente, ou um representante dele, está o tempo todo presente no desenvolvimento do sistema, e isso diminui erros e gera um produto de *software* com uma qualidade muito superior.

Os atributos de uma boa história de usuário são, de maneira resumida:

- escritas usando linguagem que possa ser facilmente entendida pelo cliente e pelo usuário final;

- curtas o suficiente para que sejam compostas por poucas frases e possam caber em um pequeno cartão (ficha ou eletrônico, como um *Trello*, por exemplo);
- focadas o suficiente para descreverem pequenos incrementos de funcionalidade, que podem ser concluídos em vários dias ou semanas. Nesse caso, estamos nos referindo a *sprints* em um processo ágil baseado em SCRUM, por exemplo;
- permitirem ser alteradas com frequência com base em discussões com o cliente, à medida que a funcionalidade se torna melhor compreendida pelo time de desenvolvimento.

Eu, como um cliente, quero poder procurar por voos entre duas cidades para ver qual deles oferece a melhor relação de preço e rotas.

Estimativa: 1.0 pontos

Prioridade: 2 –Alta

Gerenciamento

O **gerenciamento de requisitos** é o processo de capturar, avaliar e justificar os desejos e as necessidades das partes interessadas. Segundo o PMI, a partir dos escritos de Coventry (2015):

Gerenciamento de Requisitos: é um conjunto iterativo de atividades que ajudam a garantir que elicitação, documentação, refinamento e mudanças de requisitos sejam tratados adequadamente durante um ciclo de vida, visando satisfazer a missão ou necessidade geral de maneira qualitativa e para a satisfação do cliente.

Quando reunimos todos os requisitos e os validamos, podemos gerenciá-los, mas, para que isso ocorra, precisamos ter em mente o que se espera de um requisito bem definido:

- **unicamente identificável:** aborda apenas um requisito central;
- **atual:** atualizado e relevante para as necessidades do negócio e do sistema;
- **consistente:** não contradiz nenhum outro requisito capturado/elicidado;
- **compreensível:** concisamente declarado e não aberto a diferentes interpretações;
- **verificável:** a conformidade pode ser verificada através de inspeção, demonstração, teste ou análise;
- **rastreável:** o requisito pode ser rastreado a partir da necessidade originária, através do plano, até o que é entregue;
- **priorizada:** sua importância relativa é entendida.

A partir disso, devemos levar em consideração um processo para gerenciar tudo isso, não é mesmo?!

Um processo simples para gerenciamento de requisitos possui quatro etapas:

- coletar requisitos das partes interessadas;
- analisar os requisitos para procurar sobreposições, lacunas e conflitos;
- justificar os requisitos para distinguir desejos de necessidades;
- elaborar a linha de base dos requisitos com as necessidades antes de iniciar o processo de desenvolvimento de soluções. Linha de base é o que chamamos de primeira forma dos requisitos coletados e validados, isso é D0, ou seja, no dia zero da entrega dos documentos de requisitos, porque um mês depois e seguindo a linha de tempo, a cada nova linha de base “fotografada”, os requisitos terão evoluído, alguns terão sido modificados, outros retirados, e assim por diante.

Essas etapas serão realizadas de diferentes maneiras, dependendo da prática do setor e das metodologias de desenvolvimento individuais utilizadas. Por exemplo, a abordagem para o desenvolvimento de *software* usando métodos ágeis seria diferente daquela usando uma abordagem em cascata.

Documentação

O Documento de Requisitos do Sistema (DRS) define os requisitos funcionais e de desempenho no nível do sistema para uma solução de *software*. Ele deve incluir uma descrição nesse nível de todos os elementos de *software* exigidos pela solução de *software*. Portanto, o objetivo de um documento de especificação é descrever o comportamento e as diferentes funcionalidades de um aplicativo ou *software* em um ambiente específico.

Lembre-se, o desenvolvimento de uma solução deve começar de uma especificação. Se, de alguma forma, o *software* entregue não atender aos requisitos, a especificação servirá como referência e a equipe de desenvolvimento trabalhará para atender a todos os requisitos descritos. A detecção precoce de problemas na especificação leva a um gerenciamento eficaz do tempo, já que é muito mais fácil atualizar a especificação antes de qualquer desenvolvimento, do que atualizar a especificação e as funcionalidades correspondentes durante o desenvolvimento.



Conheça um documento de especificação de requisitos e seu conteúdo.

Disponível em: <https://bit.ly/306K9nY>

Rastreabilidade de Requisitos

Rastreabilidade de requisitos refere-se à capacidade de descrever e seguir a vida de um requisito, tanto para frente como para trás, portanto, desde suas origens, desenvolvimento e especificação, até sua implantação e usos subsequentes, e por todos os períodos de refinamento contínuo que ocorrem durante o ciclo de vida de desenvolvimento de qualquer *software*, assim como as interações que ocorrem em qualquer uma dessas fases. Realizar uma análise de rastreabilidade de requisitos é uma parte importante do processo de engenharia de *software*, pois garante que todos os requisitos foram considerados adequadamente durante cada fase do projeto, e que não existem brechas, gaps ou buracos de escopo no sistema após o seu desenvolvimento por causa de requisitos perdidos ou esquecidos.

Isso é uma das coisas mais graves que pode acontecer a um sistema quando ele vai ser entregue, “esquecer requisitos” é algo que o cliente normalmente não perdoa. A atividade de rastreamento também garante que todos os requisitos sejam internamente consistentes entre si e apoiem as principais partes interessadas no sistema, bem como suas metas e objetivos do negócio. Isso em poucas palavras representa um cliente feliz no final do desenvolvimento, mais que disposto a pagar pela conta do *software*.

A rastreabilidade envolve o estabelecimento de uma linha imaginária dos requisitos para itens relacionados, como casos de teste ou *releases*. Permite a documentação de dependências entre requisitos e com outros elementos importantes no projeto, por exemplo: *design* técnico, componentes de código, casos de teste, *releases*, incidentes, entre outros.

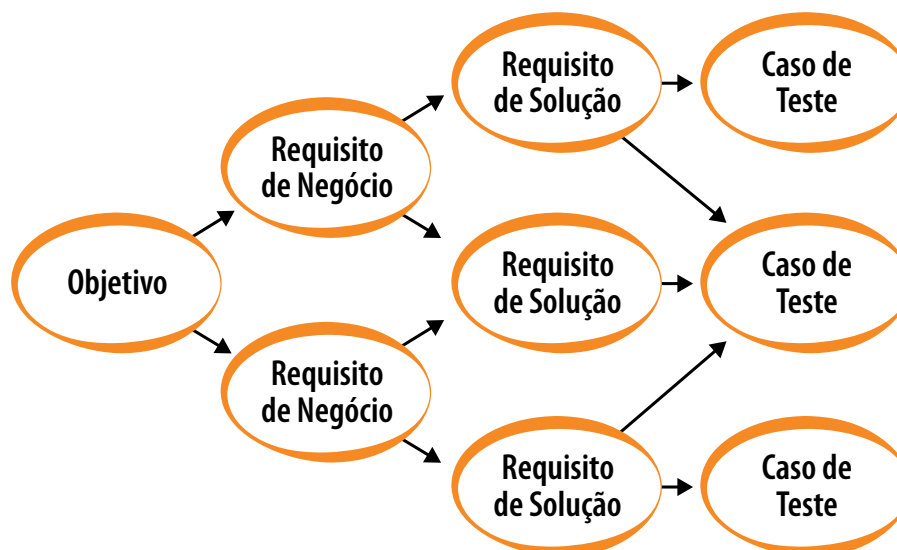


Figura 2 – Exemplo de rastreabilidade de Requisitos. “Na análise de negócios, realizamos a rastreabilidade para garantir que os requisitos sejam aprovados e gerenciados corretamente durante todo o ciclo de vida do projeto.”

Fonte: Adaptado de AZTARIAN, 2018, p. 1

Uma boa rastreabilidade de requisitos nos oferece os seguintes benefícios:

- **Gerenciar o escopo da solução do software:** porque cada requisito está associado a um identificador único e também ao objetivo do negócio ou trecho do próprio processo de negócio, isso permite que os membros do time de diagnóstico, solução ou negócios avaliem apropriadamente o valor agregado de ter esse requisito realizado no *software* ou, em comum acordo, não o ter;
- **Avaliar de maneira ágil potenciais mudanças:** durante a execução de um projeto de *software*, requisitos são transformados em funcionalidades e, em última instância, em versões de *softwares* ou nesses já prontos. Portanto, mudanças e requisições do usuário podem surgir a qualquer momento, principalmente se você estiver trabalhando com metodologias ágeis ou *lean*. Avaliar o impacto de uma mudança é questão, às vezes, de vida ou morte de um sistema. O impacto de uma possível mudança deve ser visto e decidido de maneira simples, rápida e fácil e o meio racional de fazer isso é recorrer à rastreabilidade. Dessa forma, vemos como uma mudança pode impactar em todos os outros requisitos que estão diretamente relacionados a ela, e nos que estão indiretamente também, e, sabendo com antecedência a quantidade de componentes impactados, podemos avaliar de maneira lógica se abraçamos essa mudança ou a deixamos para mais tarde, ou talvez, em outra versão da solução, elaborando um *road map* ou mapa de liberação de novas funcionalidades durante o uso da solução de *software*;
- **Reduz o risco do projeto:** quando sabemos as dependências entre requisitos e sabemos quais são os mais críticos, podemos lidar com o risco de forma proativa, o que dá vantagem para o time de desenvolvimento além da visibilidade e do melhor controle;
- **Promoção de consistência entre os requisitos:** saber qual requisito está relacionado com outro(s) requisito(s) ajuda a verificar pontos incongruentes, ligações erradas, funcionalidades inúteis e unificação de terminologia, isso evita dubiedade de interpretação e unifica a linguagem para que o time fale a “mesma língua”;
- **Monitorar e controlar durante todo o ciclo de vida do requisito:** para colocarmos todos os requisitos de uma solução de *software* em um mesmo lugar e termos controle, devemos ter em mente não apenas os requisitos aceitos e que serão embarcados, mas também os que estão pendentes e os que foram rejeitados. Você deve saber que há *softwares* que fazem tudo isso, mas a grande maioria das empresas em geral não possuem recursos para adquirir esses *softwares* porque eles são caros e requerem mão de obra especializada para operá-los. Portanto, você deverá utilizar a Matriz de rastreabilidade de requisitos, tema esse que veremos logo adiante.

Por fim, para decidir sobre o nível de rastreabilidade e o trabalho e as ferramentas necessárias, devemos considerar os seguintes aspectos, segundo Astarian (2018):

O tamanho, a complexidade e o risco do projeto: projetos maiores, complexos e arriscados exigem mais rastreabilidade.

Custo, recursos ou restrições de tempo: datas de entrega restritas podem exigir rastreabilidade mínima.

Experiência de análise de negócios dentro do projeto e organização: sistemas ou ferramentas mais complexas exigem mais experiência.

Conhecimento e disponibilidade de ferramentas de gerenciamento de requisitos: requisitos de rastreabilidade mais complexos exigem ferramentas mais avançadas e maior conhecimento.

Responsabilidade pela manutenção dos relacionamentos de rastreabilidade: sistemas complexos e frequentemente mutáveis podem exigir ferramentas mais sofisticadas, tempo da equipe e conhecimento. (AZTARIAN, 2018, p. 5)

Matriz de Rastreabilidade de Requisitos

A Matriz de Rastreabilidade de Requisitos – ou em inglês, *Requirement Traceability Matrix (RTM)* – é usada para verificar se todos os requisitos declarados e derivados estão associados aos elementos de sistema, tais como estrutura, componentes, módulos e outras entregas do projeto. Além disso, também usamos a matriz para verificar e documentar a fonte original dos requisitos, de modo que, se surgirem dúvidas do cliente ou do time de sistemas sobre o motivo pelo qual certos recursos foram incluídos/modificados, há uma trilha abrangente e precisa.

Não estamos tratando aqui de uma ferramenta complexa e que fará você ficar, literalmente, “arrancando seus cabelos”, pode-se fazer com uma planilha, uma tabela ou até mesmo em papel. Claro, se você trabalhar em um sistema muito grande e complexo, um *software* cairá muito bem; mas você deve aprender a se virar sem isso, pois o importante é entender o conceito das coisas. Um excelente analista de sistemas não depende de ferramentas!

Temos a necessidade de documentar as associações entre os requisitos funcionais e de sistema e os vários artefatos criados durante o projeto, desenvolvimento e teste do sistema, que são representados por vários artefatos que encontramos durante essas fases e que você deve saber reconhecer. Por exemplo:

- **Requisitos:** nessa fase, os casos de uso do negócio e do sistema precisam ser validados em relação aos requisitos funcionais e do sistema;
- **Design:** aqui nos interessam os elementos de *design*, modelos de objetos, modelos de dados, *designs* de módulos, diagramas de sequência, *designs* de interface do usuário e outros artefatos de *design* que devem estar relacionados aos requisitos do sistema;

- **Desenvolvimento:** aqui nos interessa que as tarefas de desenvolvimento do projeto devem estar associadas aos requisitos de origem os quais elas cumprem;
- **Testes:** aqui o importante é que a cobertura de teste de requisitos é uma métrica chave nas atividades de teste, pois garante que todos os requisitos e casos de uso tenham casos de teste de suporte que validarão a funcionalidade;
- **Manutenção:** aqui nos relacionamos com o suporte e a manutenção, todos os defeitos e problemas do sistema que são gerados precisam ser associados ao módulo e ao requisito de origem para que, além de serem corrigidos, gerem lições aprendidas.

Uma matriz de rastreabilidade de requisitos deve ser simples e funcional, seus componentes devem estar dispostos em colunas com nomes claros e devem ser orientados pelos identificadores, já que suas descrições estão no documento de requisitos da solução. Portanto, uma sugestão de cabeçalho envolve: identificador, título do requisitos do sistema, casos de uso associados, elementos de *design* e elementos de caso de testes associados. Claro, pode haver outros, mas isso é o mínimo.

Tabela 1 – Exemplo simples de uma matriz de rastreabilidade

identidade	Requisitos do sistema	Casos de uso	Elementos de design	Casos de teste
RQ1	Capacidade de criar um novo livro no catálogo	UC1, UC4, UC5	DE3, DE6	TC1, TC6, TC9
RQ2	Capacidade de editar livro existente no catálogo	UC1, UC2	DE4	TC3, TC8
RQ3	Capacidade de criar um novo autor no catálogo	UC1, UC8, UC9	DE22	TC1, TC9

Tabela 2 – Exemplo de matriz de rastreabilidade reversa mostrando de onde vieram os requisitos RQ1, 2 e 3 da Tabela 1

identidade	Requisitos do sistema	Documentos de origem	Stakeholders	Atividade de Elicitação
RQ1	Capacidade de criar um novo livro de catálogo	Lista de Metas do Projeto 1.22.2000	Bibliotecário Chefe	Workshop de Desenvolvimento de Objetivos
RQ2	Capacidade de editar livro existente no catálogo	Nenhum – implícito	Bibliotecário Joe Smith	Reunião de análise de requisitos 1.30.2000
RQ3	Capacidade de criar um novo autor no catálogo	Lista de Metas do Projeto 1.22.2000	Bibliotecário Chefe	Workshop de Desenvolvimento de Objetivos

A Matriz de rastreabilidade de requisitos é uma ferramenta multifuncional e pode ser posta sob o ponto de vista de quem está atuando em papéis específicos dentro de um time de desenvolvimento ou dos próprios usuários e analistas de negócios. Veja o arranjo informacional dessa outra forma de representar a matriz:

Tabela 3 – Matriz de rastreabilidade vista sob a perspectiva de rastrear os casos de uso

identidade	Caso de uso	Requisitos do sistema	Elementos de design	Casos de teste
UC1	Adicionando novo livro ao catálogo da biblioteca	RQ1, RQ2, RQ3	DE3, DE6	TC1
UC2	Atualizando os detalhes de um livro no sistema	RQ2	DE4	TC3
UC3	Excluindo um livro no sistema	RQ5	DE22, DE4	TC7

Diagrama de Caso de Uso

A modelagem de casos de uso é uma ferramenta útil para a eliciação de requisitos de *software* e, sem dúvida alguma, fornece uma representação gráfica dos requisitos do sistema de *software*, colocando-os na forma visual para entendimento melhor por parte dos analistas de sistemas, analistas de negócios, bem como dos engenheiros de solução.

Os elementos-chave em um modelo de caso de uso são os atores ou as entidades externas e os próprios casos de uso com os quais esses atores se envolvem e em que desempenham seus papéis. Assim sendo, um caso de uso é o que chamamos de uma unidade de funcionalidade, um serviço, ou, se preferir, um requisito. Portanto, deixaremos bem claro aqui para não deixar dúvidas que um caso de uso não é um processo, programa ou função.

Como os modelos de casos de uso são simples no conceito e na aparência, é relativamente fácil discutir a exatidão desses modelos com um cliente. A modelagem de casos de uso, mais especificamente o diagrama de caso de uso geral, foi criada em 1991 por Ivar Jacobson e foi incorporada na *UML (Unified Modeling Language)* – em português, Linguagem de Modelagem Unificada para engenharia de *software* –, sendo um dos diagramas mais utilizados até hoje.



Apenas para fins de esclarecimento, a UML é uma tentativa de universalizar a modelagem de *software* em engenharia de *software*, sendo essa iniciativa do *Object Management Group*. Caso você possua interesse, poderá acessar o site desse grupo aqui: <http://bit.ly/2FBiwdn>

Seus componentes podem ser descritos na Tabela 4 para nosso conhecimento:

Tabela 4 – Apresentação dos componentes de um diagrama de caso de uso

Símbolo	Nome	Interpretação
 Name «actor» Name	Ator	Uma entidade (humana ou não) externa ao sistema e que interage com ele.
 Name «use-case» Name	Caso de uso	Um serviço ou unidade de funcionalidade.
 Name	Limite do sistema	Indica a divisão entre o sistema que está sendo projetado e o resto do mundo.
 An Actor Name A Use Case	Associação de comunicação	A linha indica que um caso de uso específico está associado a um determinado ator. O nome é opcional e geralmente omitido. Uma seta também pode ser usada opcionalmente; quando presente, não indica um fluxo de informações (como em um diagrama de fluxo de dados).
 A Use Case Name Another Use Case	Associação de casos de uso	Indica que os casos de uso estão relacionados de uma maneira específica, por exemplo, o comportamento de um caso de uso inclui o comportamento de outro caso de uso.
 "nome"	Estereótipo	Indica que o símbolo ao qual ele está ligado pertence a uma categoria específica.
 A Use Case «name» Another Use Case Name	Generalização	Indica que os dois símbolos que ele conecta estão relacionados por uma relação de especialização de generalização. Por exemplo, um ator é um subtipo de outro ou um caso de uso é um tipo de outro. Tanto o nome quanto o estereótipo são opções.
 A Note Note	Nota	O designer pode, e deve, qualificar qualquer parte do modelo com uma nota textual se melhorar a clareza do design.

Fonte: Adaptado de cs.uct.ac.za

Todo o conjunto de **casos de uso e atores** do sistema organiza o escopo do sistema a respeito dos objetivos que os usuários atingirão quando o sistema estiver

pronto. Do ponto de vista prático, um caso de uso é uma sequência de ações executadas para obtermos um determinado objetivo, fazemos isso de forma gráfica para facilitar o entendimento e a visualização, mesmo para um leigo.

Algumas boas práticas envolvem dar o nome ao caso de uso em uma palavra ou frase indicando de forma clara o que ele realiza. A figura da elipse com rótulo na nossa Tabela 4 representa uma funcionalidade do sistema, sendo que essa pode estar estruturada em outras. Um caso de uso pode ser concreto, quando é iniciado diretamente por um ator, ou abstrato, quando é uma extensão de um outro caso de uso. Além disso, há casos de uso primários e secundários. Os primeiros representam os objetivos dos atores, já os segundos são funcionalidades do sistema que precisam existir para que esse funcione corretamente.

Como você deve ter percebido na Tabela 4, em geral uma comunicação é identificada com uma ligação sem direção, ou seja, sem setas direcionais, apenas uma linha contínua. Um caso de uso pode estar associado a mais de um ator também, sendo que os atores se dividem em ativos e passivos, em que os primeiros iniciam o caso enquanto os segundos participam do caso, porém, sem iniciá-lo.

Os relacionamentos nos casos de uso auxiliam na descrição, podendo ser entre um ator e um caso de uso, entre atores e entre casos de uso. Descreveremos um pouco sobre os tipos de relacionamento, dando alguns exemplos:

Conforme os escritos de Vieira (2015), quando falamos de casos de uso, precisamos ter em mente o conceito de cenário, chamamos isso de instâncias de caso de uso. Um cenário pode ser compreendido como uma sequência de passos que descreve uma interação entre um usuário e o sistema.

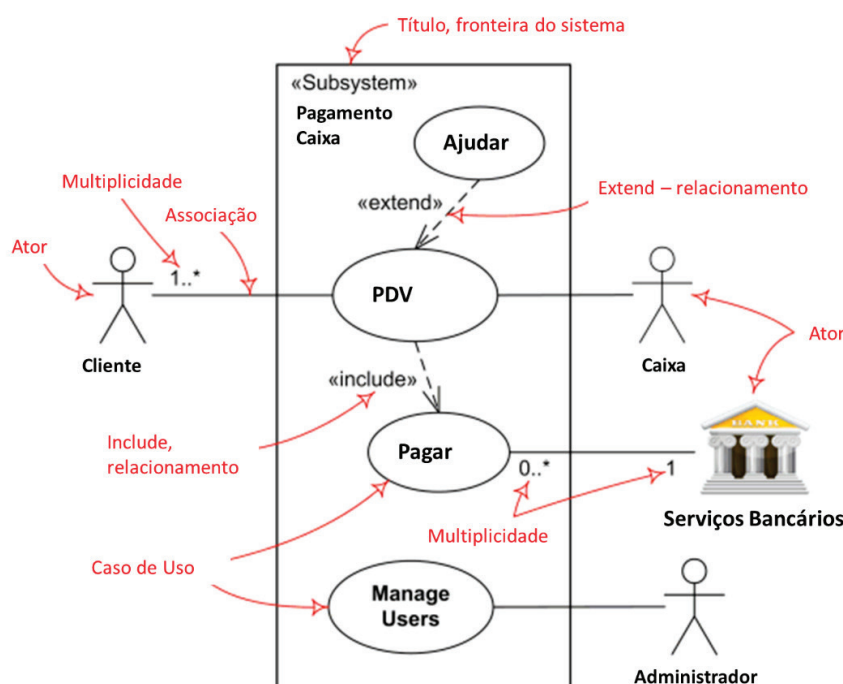


Figura 3 – Típico cenário de um diagrama de Casos de Uso

Fonte: Adaptado de VIEIRA, 2015, p. 1

Segundo Vieira (2015, p. 3-4):

- **Relacionamento de Comunicação ou Associação:** representa a interação entre um ator e um caso de uso por meio de mensagens. É representado por uma linha sólida:



Figura 4

Fonte: Adaptado de VIEIRA, 2015, p. 3-4

- **Relacionamento de Inclusão:** utilizado quando um comportamento se repete em mais de um caso de uso. Por exemplo, em um *internet banking*, um cliente que vai realizar um pagamento precisa se logar, assim como um cliente que vai visualizar o saldo também precisa se logar:

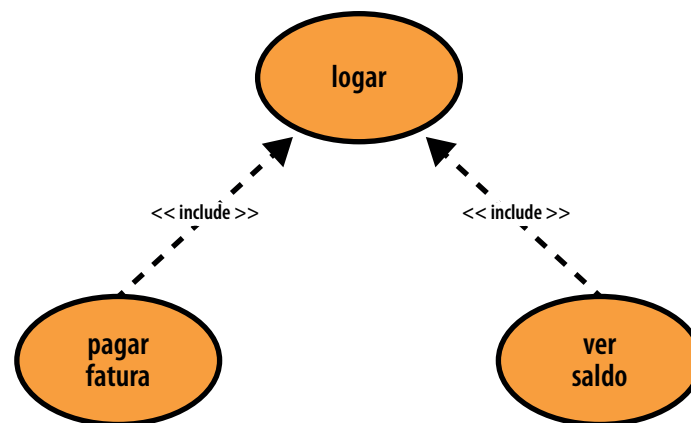


Figura 5

Fonte: Adaptado de VIEIRA, 2015, p. 3-4

- **Relacionamento de Extensão:** utilizado quando se deseja modelar um relacionamento alternativo. Por exemplo, ao “cadastrar usuário” num sistema de forum, podemos “cadastrar um administrador” ou “cadastrar um moderador”:

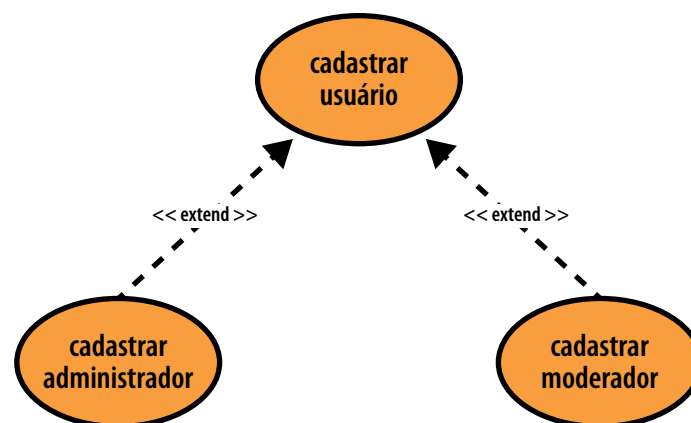


Figura 6

Fonte: Adaptado de VIEIRA, 2015, p. 3-4

- **Relacionamento de Herança:** é um relacionamento entre atores, utilizado quando queremos representar uma especialização/generalização. Na figura a seguir, vendedor é especialização de pessoa (ou pessoa é generalização de vendedor), é representado por um alinhamento com um triângulo:

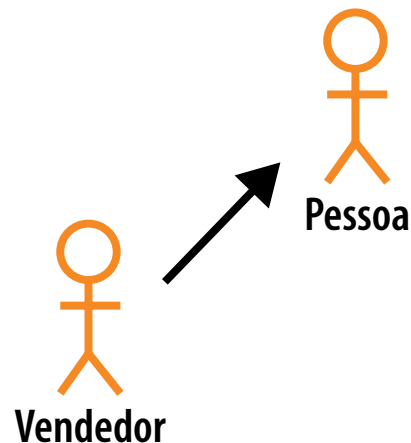


Figura 7

Fonte: Adaptado de VIEIRA, 2015, p. 3-4

Talvez você esteja se perguntando: mas como eu junto tudo isso e coloco nesse diagrama de forma que as pessoas entendam o que eu elicitei de requisitos e possamos todos discutir e agregar mais valor partindo de um ponto inicial, o qual é esse importante diagrama?

Trabalharemos em um exemplo prático excelente nesse caso de negócio, um excerto de Ribeiro (2012):

A clínica médica Saúde Perfeita precisa de um sistema de agendamento de consultas e exames. Um paciente entra em contato com a clínica para marcar consultas visando realizar um check-up anual com seu médico de preferência. A recepcionista procura data e hora disponível mais próxima na agenda do médico e marca as consultas. Posteriormente o paciente realiza a consulta, e nela o médico pode prescrever medicações e exames, caso necessário.

Com esse cenário simples podemos começar a criar nosso diagrama. Inicialmente vamos definir nossos atores: Paciente, Secretária, Médico.

Agora vamos definir algumas ações de cada usuário:

- **Paciente:** (Solicitar Consulta, Solicitar Cancelamento de Consulta);
- **Secretária:** (Consultar Agenda, Marcar Consulta, Cancelar Consulta);
- **Médico:** (Realizar Consulta, Prescrever Medicação e Solicitar realização de exames).

Vamos traduzir isso em um diagrama de caso de uso de forma a todos poderem ver e entender graficamente quem acessa o que através das necessidades dos atores. (RIBEIRO, 2012, p. 6-7)

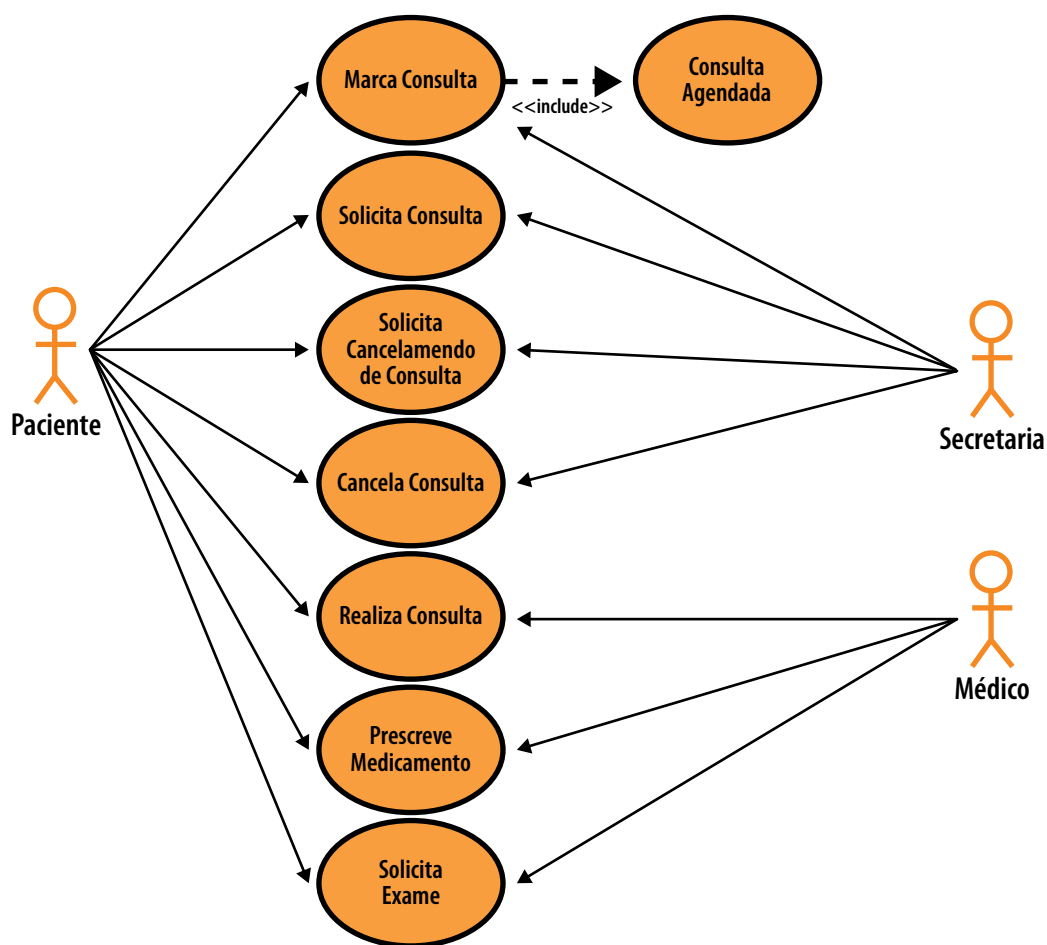


Figura 8 – Solução proposta para o problema descrito quanto a representação dos requisitos do cliente representado pelos atores e suas interações em um diagrama de caso de uso

Fonte: Adaptado de RIBEIRO, 2012, p. 6-7

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

Entenda o diagrama de casos de uso, 2015

Utiliza o ASTAH para desenhar o diagrama e permite você aprender um pouco sobre essa ferramenta de desenho de diagramas.

<https://youtu.be/xrcgbMQdM8Y>

Eles não lêem o documento de requisitos

<https://youtu.be/zAyVVWeQGNU>

Leitura

Documento de requisitos: sistema de aquisição e processamento automático de informações voluntárias de enchentes de redes sociais (SOCIAL-FLOOD)

<http://bit.ly/2s7BJQE>

Especificação e documentação de requisitos: um modelo aplicável à análise da informação utilizando “casos de uso”

<http://bit.ly/36FkGo2>

Referências

AZTARIAN, A. M. **Rastreabilidade de requisitos** – O que, por que e como. 2018. Disponível em: <<https://www.b2ttraining.com/requirements-traceability/>>. Acesso em: 30 jul. 2019.

COVENTRY, T. Gerenciamento de requisitos, planejamento para o sucesso! – técnicas para acertar ao planejar requisitos. PMI® GLOBAL CONGRESS, 2015, Londres. **Anais [...]**. Newtown Square, PA: Project Management Institute, 2015. Disponível em: <<https://www.pmi.org/learning/library/requirements-management-planning-for-success-9669>>. Acesso em: 30 jul. 2019.

RIBEIRO, L. **O que é UML e Diagramas de Caso de Uso**: introdução prática à UML. 2012. Disponível em: <<https://www.devmedia.com.br/o-que-e-uml-e-diagramas-de-caso-de-uso-introducao-pratica-a-uml/23408>>. Acesso em: 30 jul. 2019.

SEBOK. **Guide to the System Engineering Body of Knowledge**. 2011. Disponível em: <[https://www.sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_\(SEBoK\)](https://www.sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK))>. Acesso em: 30 jul. 2019.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

VIEIRA, R. **UML** – Diagrama de Caso de Uso. 2015. Disponível em: <<https://medium.com/operacionalti/uml-diagrama-de-casos-de-uso-29f4358ce4d5>>. Acesso em: 30 jul. 2019.

WAZLAWICK, R. S. **Análise e projeto orientado a objetos para sistemas de informação-modelando com UML, OCL e IFML**. São Paulo: ELSEVIER, 2014.



Cruzeiro do Sul
Educatonal